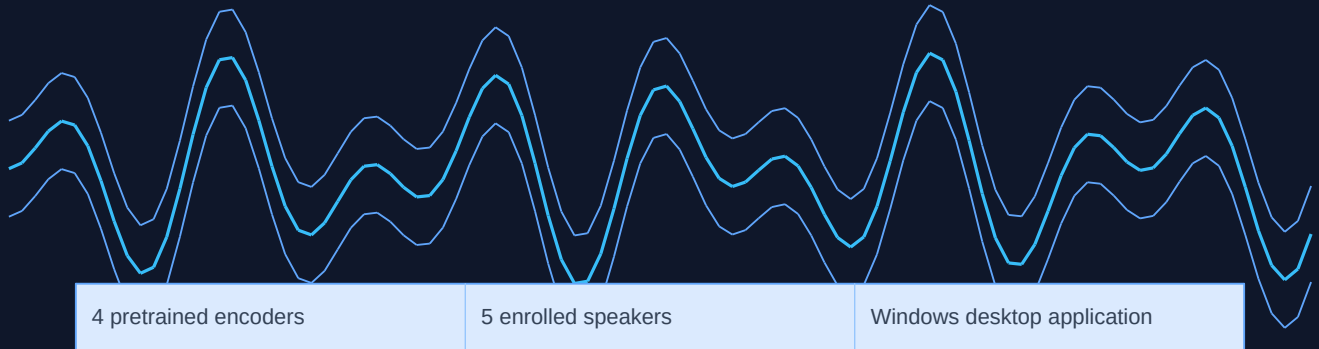


MULTI-MODEL SPEAKER IDENTIFICATION SYSTEM

Technical Project Documentation



Pretrained speaker embeddings with per-model SVM classification, probability analysis, signal visualisation and multi-model PDF reporting

Version 1.0 | 24 July 2026

Table of Contents

Document Control4

1. Project Overview5

1.1 Objective5

1.2 Current Scope5

1.3 Primary Outputs5

2. System Architecture6

2.1 Component Responsibilities6

2.2 Offline and Online Paths6

3. Dataset and Audio Preprocessing7

3.1 Enrolled Classes7

3.2 Dataset Layout7

3.3 Preprocessing Pipeline7

3.4 Non-Speaker Exclusion7

4. Pretrained Model Portfolio8

4.1 SpeechBrain ECAPA-TDNN8

4.2 SpeechBrain X-Vector8

4.3 Microsoft WavLM Base Plus SV8

4.4 Microsoft UniSpeech-SAT Base Plus SV8

5. Embedding Extraction and Data Quality9

5.1 Validation and Failure Handling9

5.2 Generated Files9

6. Classifier Training Methodology10

6.1 Standard Multi-Model Training10

6.2 ECAPA Legacy Classifier10

6.3 Saved Artifacts10

7. Experimental Results11

7.1 Interpretation11

7.2 Best-Model Confusion Matrix12

8. Online Prediction and Confidence13

8.1 Inference Sequence13

8.2 Return Contract13

8.3 Correct Confidence Interpretation 13

9. Desktop Application 14

9.1 Dashboard 14

9.2 Waveform Page 14

9.3 Spectrogram Page 14

9.4 Settings Page 14

9.5 Sidebar Export Action 14

10. Complete Multi-Model PDF Report 15

10.1 Export Algorithm 15

10.2 Probability Integrity 15

10.3 Report Contents by Model 15

11. Installation and Operation 16

11.1 Create the Environment 16

11.2 Prepare Dataset and Train a Model 16

11.3 Available Model IDs 16

11.4 Launch the Desktop Application 16

11.5 Normal Application Workflow 16

12. Project Structure and Maintenance 17

12.1 Adding Another Encoder 17

12.2 Configuration Consistency 17

12.3 Backup Practice 17

13. Troubleshooting Guide 18

13.1 Recommended Diagnostics 18

14. Limitations and Future Work 19

14.1 Current Limitations 19

14.2 Recommended Enhancements 19

14.3 Conclusion 19

15. References 20

Document Control

Item	Value
Document	Technical Project Documentation
Project version	0.1.0 application / 1.0 documentation
Platform	Windows desktop, Python 3.12 environment
Documentation date	24 July 2026
Project root	C:\Users\addld\Desktop\speaker_identification_01
Evidence basis	Current source code, saved embeddings, model artifacts, metrics JSON files and exported reports.

1. Project Overview

This project is a closed-set speaker identification system. It accepts a speech recording, converts it into a speaker embedding with a selected pretrained encoder, and uses a speaker-specific classifier to identify the most likely person among the enrolled classes.

1.1 Objective

- Identify who is speaking rather than transcribing what is said.
- Compare multiple pretrained speaker-recognition backends in one application.
- Return a ranked probability distribution across all enrolled speakers.
- Provide waveform and Mel spectrogram inspection tools.
- Export one consolidated PDF with audio details and every trained model result.

1.2 Current Scope

Dimension	Current implementation
Identification type	Closed-set, single-speaker classification
Enrolled speakers	5
Pretrained encoders	4 active configurations
Input sample rate	16,000 Hz
Embedding size	192 or 512 values, model-dependent
Classifier	Per-model RBF SVM with probability output
Interface	CustomTkinter multi-page Windows desktop application
Reports	Per-model pages plus waveform and spectrogram

Important design point: the pretrained neural encoders are not retrained on this dataset. They are used as fixed feature extractors. The project trains a separate downstream classifier for each embedding space.

1.3 Primary Outputs

- Predicted speaker label.
- Calibrated or model-provided confidence percentage.
- Probability assigned to each enrolled speaker.
- Inference time in milliseconds.
- Waveform and Mel spectrogram visualisations.
- Complete multi-model PDF report.

2. System Architecture

The application separates offline model preparation from online prediction. Each encoder has its own embedding matrix, scaler, classifier and label encoder. This prevents feature spaces from being mixed.

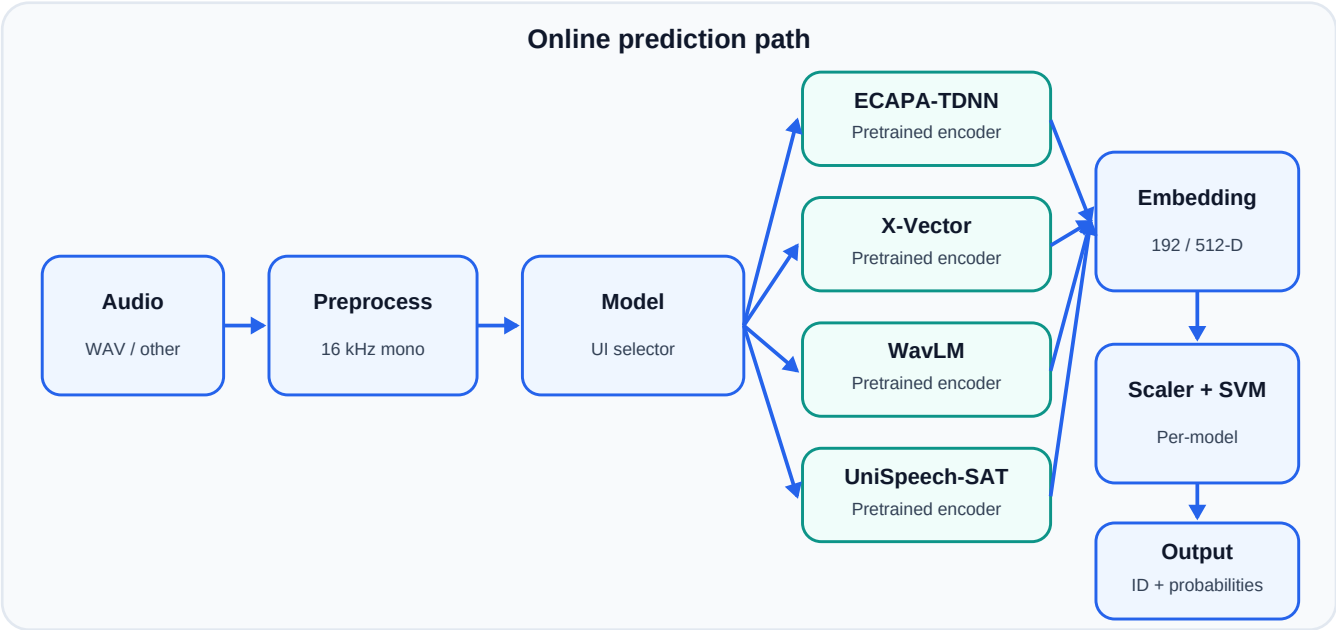


Figure 1. Online multi-model speaker identification path.

2.1 Component Responsibilities

Component	Responsibility
audio_processing.py	Standardises dataset audio before embedding extraction.
model_config.py	Registers model identifiers, sources, directories and backends.
embedding_extractors.py	Loads one pretrained encoder, prepares audio and returns a validated one-dimensional embedding.
extract_embeddings_multi.py	Scans speaker folders, skips non-speaker classes, extracts embeddings and records failures.
train_model_multi.py	Fits the per-model scaler, calibrated SVM and label encoder.
predict_multi.py	Loads cached artifacts, extracts one embedding and returns speaker, confidence, time and full probabilities.
ui/	Provides dashboard, plots, settings, navigation and complete PDF reporting.

2.2 Offline and Online Paths

- Offline: preprocess dataset, extract embeddings, split data, train and save one classifier per encoder.
- Online: select model, load audio, extract embedding, apply that model's scaler and obtain probabilities.
- Report export: repeat online prediction for every trained model and compile a single PDF.

3. Dataset and Audio Preprocessing

Speaker identity is derived from the folder name. Each speaker folder contains individual audio segments. Two non-speaker folders are explicitly excluded during embedding extraction.

3.1 Enrolled Classes

Stored class label	Audio files
Benjamin_Netanyau	1500
Jens_Stoltenberg	1500
Julia_Gillard	1501
Magaret_Tarcher	1500
Nelson_Mandela	1500
Total	7501

The stored labels preserve the existing folder spellings (for example Benjamin_Netanyau and Magaret_Tarcher). A future display-name mapping can correct presentation without renaming trained classes.

3.2 Dataset Layout

```
dataset/  
  Benjamin_Netanyau/  
  Jens_Stoltenberg/  
  Julia_Gillard/  
  Magaret_Tarcher/  
  Nelson_Mandela/  
  _background_noise_/      # excluded from identification  
  other/                   # excluded from identification
```

3.3 Preprocessing Pipeline

Step	Implementation	Purpose
Decode	librosa.load()	Read WAV, MP3, FLAC, OGG or M4A.
Resample	16,000 Hz	Match pretrained speaker models.
Channels	Mono	Provide one waveform channel.
Silence trim	top_db=20	Remove leading and trailing quiet regions.
Noise reduction	Disabled by default	Avoid unnecessary voice distortion.
Normalisation	Peak normalisation	Keep amplitudes in a consistent range.
Output	WAV at 16 kHz	Create processed_dataset/.

3.4 Non-Speaker Exclusion

extract_embeddings_multi.py skips _background_noise_ and other. They are useful as environmental data but must not be treated as speaker identities in the present closed-set classifier.

4. Pretrained Model Portfolio

The application currently exposes four pretrained speaker encoders. The source model converts audio into a fixed-size speaker representation; the project classifier operates on that representation.

Model	Provider	Architecture	Source	Dim.
SpeechBrain - ECAPA-TDNN	SpeechBrain	ECAPA-TDNN	speechbrain/spkrec-ecapa-voxceleb	192
SpeechBrain - X-Vector	SpeechBrain	X-Vector TDNN	speechbrain/spkrec-xvect-voxceleb	512
Microsoft - WavLM Base Plus SV	Microsoft	WavLM Transformer	microsoft/wavlm-base-plus-sv	512
Microsoft - UniSpeech-SAT Base Plus SV	Microsoft / Hugging Face	UniSpeech-SAT + X-Vector Head	microsoft/unispeech-sat-base-plus-sv	512

4.1 SpeechBrain ECAPA-TDNN

ECAPA-TDNN uses convolutional and residual components with attentive statistical pooling. The project obtains a 192-dimensional embedding from the SpeechBrain VoxCeleb model.

4.2 SpeechBrain X-Vector

The X-Vector backend uses a TDNN with statistical pooling to create a 512-dimensional utterance-level speaker embedding. It produced the highest current test accuracy in this project.

4.3 Microsoft WavLM Base Plus SV

WavLM is a self-supervised Transformer-based speech model designed to preserve speaker and content information. The speaker-verification checkpoint provides a 512-dimensional X-Vector-style embedding.

4.4 Microsoft UniSpeech-SAT Base Plus SV

UniSpeech-SAT applies speaker-aware self-supervised pretraining and an X-Vector head. The project uses its 512-dimensional normalised speaker embedding.

All four configured pretrained checkpoints expect 16 kHz speech. The application therefore standardises audio to 16 kHz before model inference.

5. Embedding Extraction and Data Quality

Embedding extraction is executed one model at a time so that output arrays remain homogeneous. Each successful waveform produces one row in X.npy and one speaker label in y.npy.

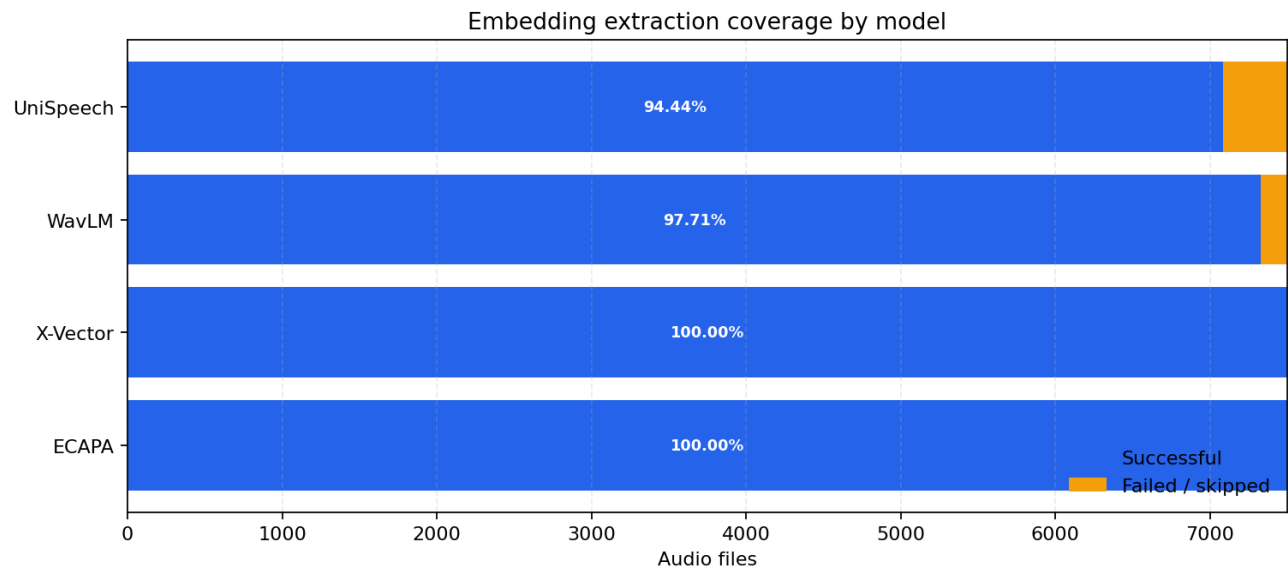


Figure 2. Successful embeddings and failed or skipped audio files.

Model	Shape	Success	Failed	Coverage
SpeechBrain - ECAPA-TDNN	(7501, 192)	7501	0	100.00%
SpeechBrain - X-Vector	(7501, 512)	7501	0	100.00%
Microsoft - WavLM Base Plus SV	(7329, 512)	7329	172	97.71%
Microsoft - UniSpeech-SAT Base Plus SV	(7084, 512)	7084	417	94.44%

5.1 Validation and Failure Handling

- Reject empty embeddings.
- Reject NaN or infinite values.
- L2-normalise model outputs where required.
- Record file path, summary and detailed exception in failed_files.txt.
- Continue the dataset run instead of aborting on one corrupt or too-short clip.

WavLM and UniSpeech-SAT lost some samples because very short processed clips can become smaller than early convolution kernels, or can yield invalid embeddings. This explains why their training sets are smaller than the complete 7,501-file dataset.

5.2 Generated Files

```
embeddings/<model_id>/
X.npy           # float embedding matrix
y.npy           # speaker labels
audio_paths.npy # successful source files
metadata.json   # model, counts, dimension and timestamp
failed_files.txt # only created when failures occur
```

6. Classifier Training Methodology

Each encoder receives an independent classifier. Training uses only the saved embeddings; the large pretrained neural model is not updated.

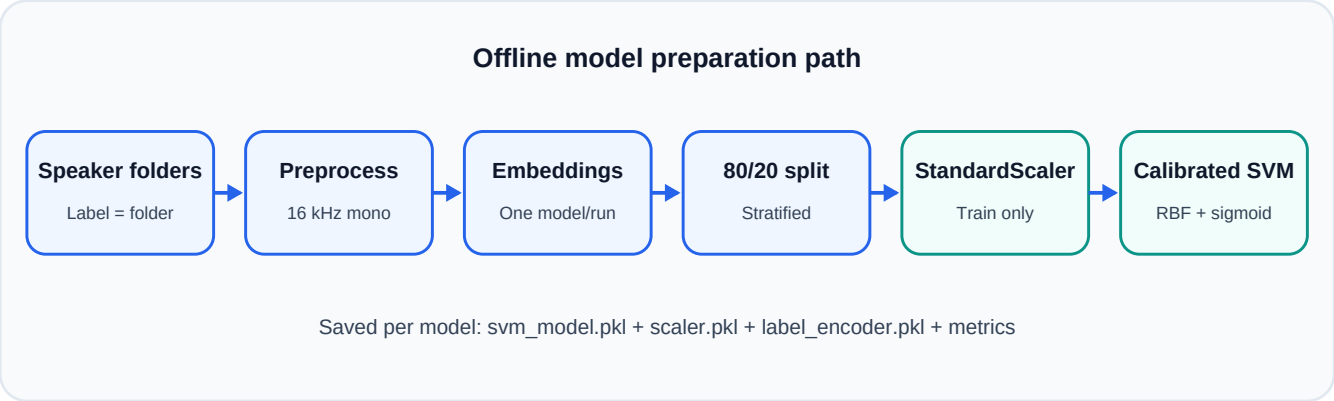


Figure 3. Per-model classifier training workflow.

6.1 Standard Multi-Model Training

- Encode folder labels with LabelEncoder.
- Create a stratified 80% training and 20% testing split.
- Use random_state=42 for repeatability.
- Fit StandardScaler on training embeddings only.
- Train an RBF SVC with C=10 and gamma='scale'.
- Wrap the SVC in CalibratedClassifierCV with sigmoid calibration and 5-fold CV.
- Verify that every probability vector sums to one.

6.2 ECAPA Legacy Classifier

The current ECAPA artifact predates the shared multi-model trainer. It is an RBF SVC with C=1 and probability=True. The other three active model artifacts were produced by train_model_multi.py and use explicit calibration.

6.3 Saved Artifacts

Artifact	Purpose
svm_model.pkl	Classifier and probability calibration state.
scaler.pkl	Feature mean and scale fitted on training data.
label_encoder.pkl	Mapping between numeric classes and speaker names.
confusion_matrix.npy	Model evaluation confusion matrix.
classification_report.txt	Per-class precision, recall and F1.
metrics.json	Model configuration, split counts and aggregate metrics.

7. Experimental Results

The following values were read from the current saved model artifacts. X-Vector, WavLM and UniSpeech-SAT use their metrics JSON files. ECAPA was recomputed from its saved test split.

Model	Test	Accuracy	Weighted precision	Weighted recall	Weighted F1
SpeechBrain - ECAPA-TDNN	1501	88.67%	89.33%	88.67%	88.85%
SpeechBrain - X-Vector	1501	91.94%	91.99%	91.94%	91.95%
Microsoft - WavLM Base Plus SV	1466	88.40%	88.53%	88.40%	88.43%
Microsoft - UniSpeech-SAT Base Plus SV	1417	89.70%	89.86%	89.70%	89.75%

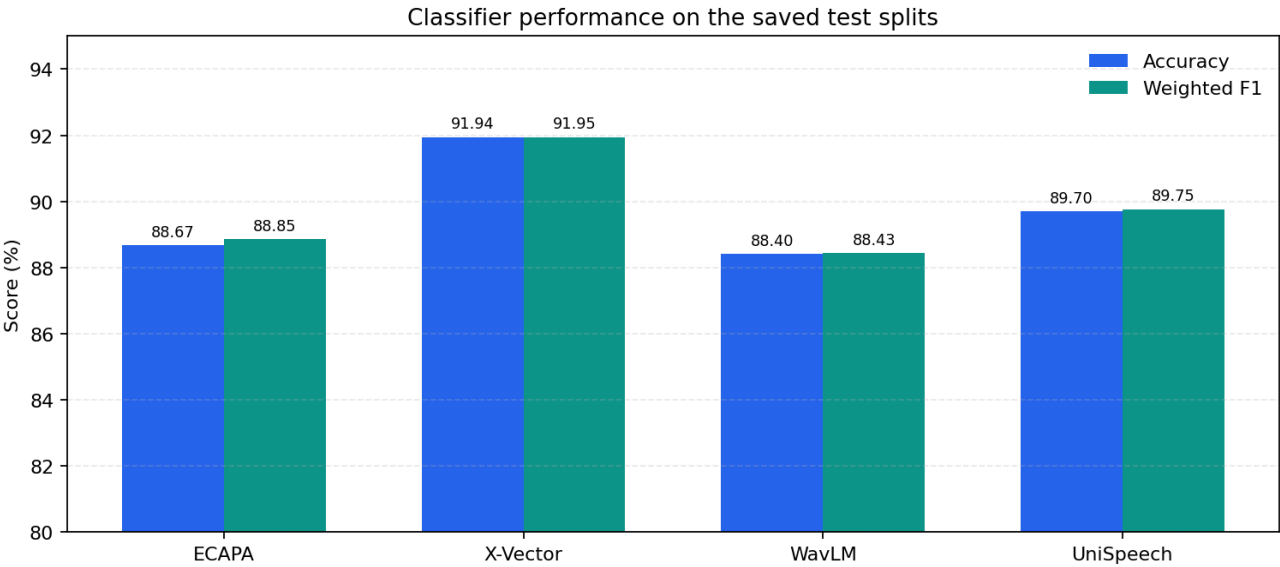


Figure 4. Accuracy and weighted F1 comparison.

Current best: SpeechBrain - X-Vector achieved 91.94% accuracy and 91.95% weighted F1 on its saved stratified test split.

7.1 Interpretation

- X-Vector provides the strongest aggregate result on this dataset.
- UniSpeech-SAT ranks second despite losing more very-short files during extraction.
- The evaluation sets differ slightly for WavLM and UniSpeech-SAT because failed clips are absent.
- Results are from one stratified split, so they should not be presented as cross-validation averages.

7.2 Best-Model Confusion Matrix

The SpeechBrain X-Vector confusion matrix shows how the 1,501 test samples were distributed across true and predicted classes.

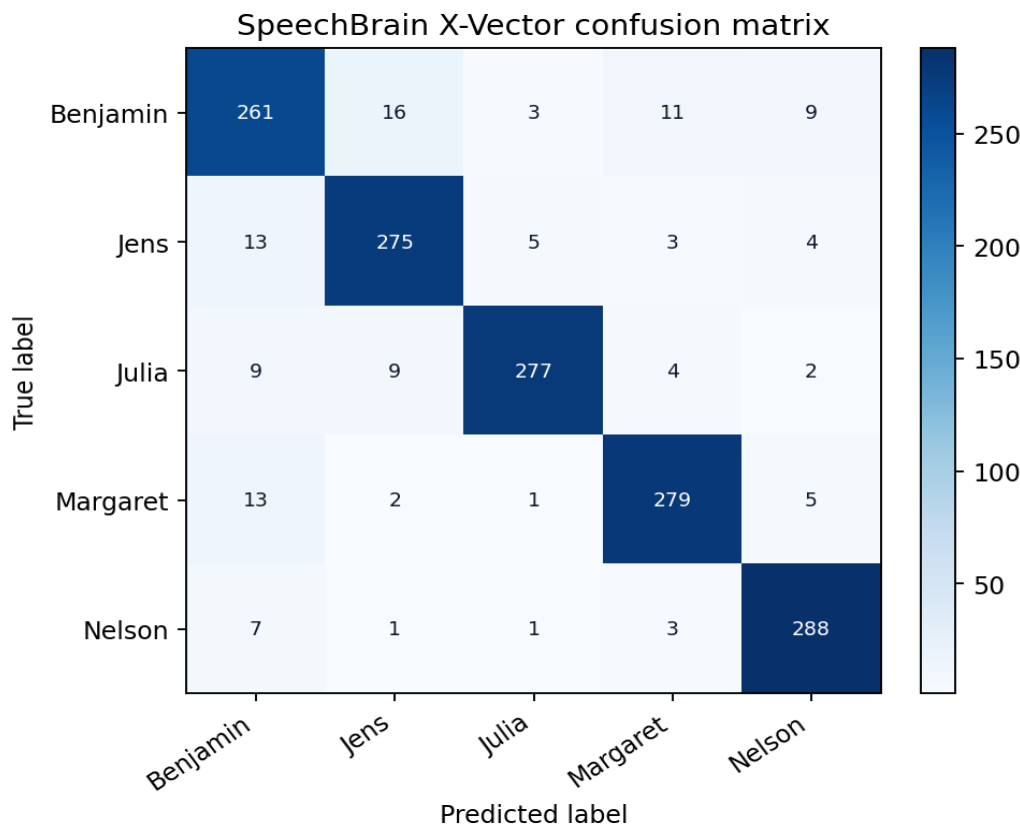


Figure 5. Confusion matrix for the current best-performing X-Vector classifier.

Speaker	Precision	Recall	F1	Support
Benjamin_Netanyau	0.86	0.87	0.87	300
Jens_Stoltenberg	0.91	0.92	0.91	300
Julia_Gillard	0.97	0.92	0.94	301
Magaret_Tarcher	0.93	0.93	0.93	300
Nelson_Mandela	0.94	0.96	0.95	300

8. Online Prediction and Confidence

The prediction path reuses cached neural models and classifier artifacts whenever possible. This reduces repeated loading cost when the same backend is used more than once.

8.1 Inference Sequence

Step	Operation
1	Validate the audio path and read the selected model configuration.
2	Load the classifier, scaler and label encoder for that model.
3	Load and prepare the waveform at the configured sample rate.
4	Extract and validate one speaker embedding.
5	Confirm that embedding dimension matches the saved scaler.
6	Apply <code>StandardScaler.transform()</code> .
7	Call <code>classifier.predict_proba()</code> .
8	Normalise small floating-point differences so probabilities sum to 100%.
9	Decode labels, sort probabilities and return the highest-ranked speaker.

8.2 Return Contract

```
speaker, confidence, inference_time_ms, probabilities = predict_speaker(
    audio_path,
    model_id="speechbrain_xvector",
)

# probabilities:
# [("Nelson_Mandela", 94.20), ("Julia_Gillard", 2.40), ...]
```

8.3 Correct Confidence Interpretation

A result such as Nelson_Mandela = 98% means the classifier assigned the remaining 2% across the other enrolled classes. It does not prove that 2% of the recording belongs to another person, and it is not a calibrated real-world identity guarantee.

Closed-set limitation: the current backend always returns one of the five enrolled labels. The confidence threshold visible in Settings is an integration hook; `run_prediction()` does not yet replace low-confidence results with an Unknown label.

9. Desktop Application

The CustomTkinter interface uses one persistent sidebar and multiple stacked pages. Navigation raises the selected frame without rebuilding the whole application.

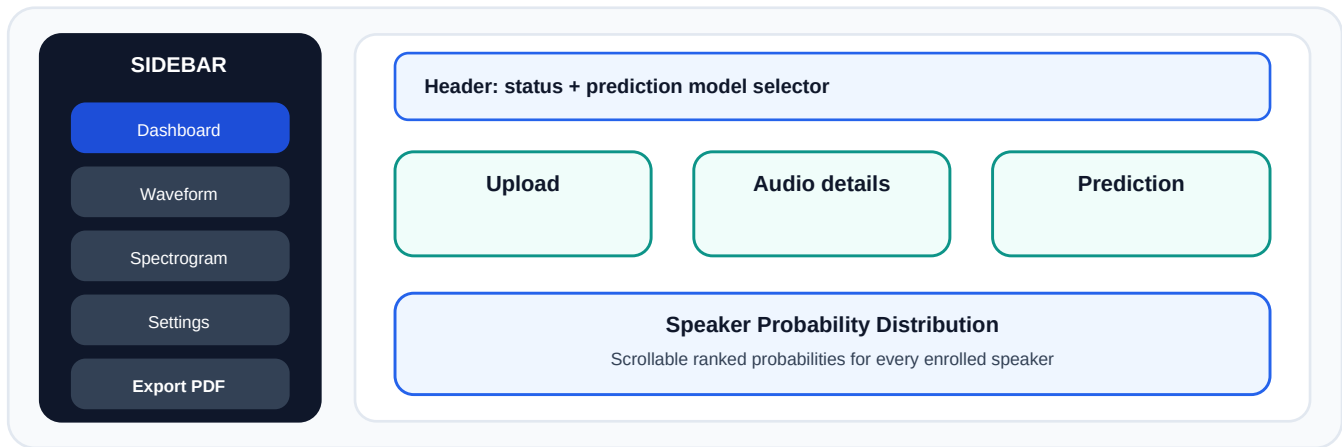


Figure 6. Current dashboard and sidebar organisation.

9.1 Dashboard

- Upload an audio file and display filename and audio metadata.
- Choose any trained embedding model from the model selector.
- Display predicted speaker, inference time and confidence meter.
- Show a scrollable probability row for every trained speaker.

9.2 Waveform Page

- Large time-domain waveform with time and amplitude axes.
- Matplotlib toolbar for zoom, pan and reset.
- Duration, sample rate, peak amplitude and RMS statistics.
- Save as PNG and export as PDF.

9.3 Spectrogram Page

- Large Mel spectrogram with dB colour scale.
- Selectable colormaps including magma, inferno, plasma and viridis.
- Audio and spectrogram statistics.
- Save as PNG and export as PDF.

9.4 Settings Page

- Dark, light or system appearance mode.
- Accent colour preview and selection.
- Default sample rate and audio-processing switches.
- Prediction threshold setting and spectrogram colormap.
- Export folder selection and reset-to-defaults action.

9.5 Sidebar Export Action

Export Complete PDF appears below Settings as an action, not a page. Clicking it retains the active page and opens the save dialog for the complete multi-model report.

10. Complete Multi-Model PDF Report

The complete export runs the selected audio through every trained model and creates a single aligned A4 document. A failure in one model is recorded on its page without cancelling the remaining report.

Page group	Content
Opening page	Audio details and comparison summary for every trained model.
One page per model	Provider, architecture, source, embedding dimension, classifier, prediction, confidence, inference time and speaker probability distribution.
Final visual page	Waveform and Mel spectrogram together on the same page.

10.1 Export Algorithm

- Find all models with complete classifier artifacts.
- Predict the same audio with each trained model.
- Store successful results or an error message per model.
- Read the selected spectrogram colormap from Settings.
- Generate waveform and Mel spectrogram images in a temporary directory.
- Build the PDF with a reusable header, footer and page numbering.
- Delete temporary plot images after the document is complete.

10.2 Probability Integrity

The PDF normalises supplied probabilities when necessary and sorts them in descending order. This avoids the earlier display error where independently transformed percentages could add up to more than 100%.

10.3 Report Contents by Model

Model page	Primary details
SpeechBrain - ECAPA-TDNN	192-D embedding; RBF SVC, C=1, probability=True; full five-speaker distribution.
SpeechBrain - X-Vector	512-D embedding; Calibrated RBF SVM, C=10.0, sigmoid calibration, 5-fold CV; full five-speaker distribution.
Microsoft - WavLM Base Plus SV	512-D embedding; Calibrated RBF SVM, C=10.0, sigmoid calibration, 5-fold CV; full five-speaker distribution.
Microsoft - UniSpeech-SAT Base Plus SV	512-D embedding; Calibrated RBF SVM, C=10.0, sigmoid calibration, 5-fold CV; full five-speaker distribution.

11. Installation and Operation

The project is configured for Windows with Python 3.12. Directly invoking the virtual environment's Python executable avoids PowerShell activation-policy errors.

11.1 Create the Environment

```
cd C:\Users\addld\Desktop\speaker_identification_01
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

11.2 Prepare Dataset and Train a Model

```
.\.venv\Scripts\python.exe audio_processing.py

.\.venv\Scripts\python.exe extract_embeddings_multi.py `
    --model speechbrain_xvector

.\.venv\Scripts\python.exe train_model_multi.py `
    --model speechbrain_xvector
```

11.3 Available Model IDs

```
speechbrain_ecapa
speechbrain_xvector
wavlm_base_plus_sv
unispeech_sat_base_plus_sv
```

11.4 Launch the Desktop Application

```
.\.venv\Scripts\python.exe app.py
```

11.5 Normal Application Workflow

- Open Dashboard.
- Choose the prediction model from the header selector.
- Browse and select a WAV file.
- Review audio details, predicted speaker and all probabilities.
- Open Waveform or Spectrogram for detailed inspection.
- Use Export Complete PDF below Settings for the full report.

The first use of a pretrained backend may download model files from Hugging Face. Later runs use the local cache.

12. Project Structure and Maintenance

The codebase separates data preparation, model extraction, classifier training, prediction and user-interface concerns.

```
speaker_identification_01/  
  app.py  
  audio_processing.py  
  model_config.py  
  embedding_extractors.py  
  extract_embeddings_multi.py  
  train_model_multi.py  
  predict_multi.py  
  dataset/  
  processed_dataset/  
  embeddings/<model_id>/  
  models/<model_id>/  
  exports/  
  ui/  
    dashboard_page.py  
    sidebar.py  
    waveform_page.py  
    spectrogram_page.py  
    settings_page.py  
    pdf_report.py  
    cards.py  
    meter.py  
    theme.py
```

12.1 Adding Another Encoder

- Add a complete entry to MODEL_CONFIGS.
- Implement or connect its loader and extractor in embedding_extractors.py.
- Return a finite one-dimensional embedding with stable dimension.
- Run extraction and training with the new model ID.
- The dashboard selector and complete report discover trained models automatically.

12.2 Configuration Consistency

requirements.txt and pyproject.toml currently disagree on some package versions, including SpeechBrain, and pyproject.toml still lists TorchCodec. Select one canonical dependency manifest and keep both files synchronised before releasing or reproducing the project.

12.3 Backup Practice

Before replacing UI or model files, copy the originals to a timestamped backups/ directory. Classifier artifacts and embedding arrays should also be backed up because regenerating them can take substantial time.

13. Troubleshooting Guide

The following issues were encountered during development and are relevant to reproducing the current Windows environment.

Symptom	Cause	Recommended action
Activate.ps1 is blocked	PowerShell execution policy	Run .venv\Scripts\python.exe directly; activation is optional.
WinError 1314 during model download	Windows symlink privilege	Use SpeechBrain LocalStrategy.COPY or enable Developer Mode.
use_auth_token argument error	SpeechBrain / huggingface_hub API mismatch	Pin a compatible pair or update the compatibility layer.
TorchCodec DLL cannot load	Python or TorchCodec binary mismatch	Use the Python 3.12 environment and avoid unnecessary TorchCodec dependency.
Kernel size greater than input size	Audio segment is too short for Transformer front-end	Pad to a minimum duration or record the failure and continue.
Embedding contains invalid values	Silent, damaged or numerically unstable audio	Validate waveform length and energy; skip the clip if still invalid.
No trained model appears in UI	Missing classifier, scaler or label encoder	Run extract_embeddings_multi.py and train_model_multi.py for that model.
Probabilities total above 100%	Values were transformed independently	Use the normalised full predict_proba vector returned by predict_multi.py.

13.1 Recommended Diagnostics

```

.\venv\Scripts\python.exe -c "import sys; print(sys.version)"
.\venv\Scripts\python.exe -m pip show torch torchaudio speechbrain
.\venv\Scripts\python.exe -m pip show huggingface_hub transformers

```

14. Limitations and Future Work

The application is a strong experimental desktop system, but several changes are required before it should be treated as a production biometric identity service.

14.1 Current Limitations

- Closed-set output: an unenrolled voice is still assigned to one of five speakers.
- Only one 80/20 split is reported; cross-validation and external test data are not yet used.
- Audio segments lost by one encoder create slightly different evaluation sets across models.
- Confidence threshold exists in Settings but is not yet enforced in dashboard prediction.
- The application processes one primary speaker per file and does not perform diarisation.
- Folder labels contain spelling inconsistencies that propagate into reports.
- Model confidence must not be interpreted as legal or forensic identity proof.

14.2 Recommended Enhancements

Priority	Enhancement	Expected value
High	Implement Unknown / Low confidence rejection	Avoid forced identification of unenrolled voices.
High	Create a shared valid-file evaluation set	Make model comparison directly fair.
High	Synchronise dependency manifests	Improve reproducibility.
Medium	Add grouped or speaker-independent cross-validation	Produce more reliable research metrics.
Medium	Pad or merge very short clips	Reduce Transformer extraction failures.
Medium	Add human-readable display-name mapping	Correct presentation without changing stored labels.
Medium	Persist settings to JSON	Retain theme, threshold and export preferences.
Long term	Add NeMo model configuration	Use the existing NeMo extractor extension point.
Long term	Package with PyInstaller	Simplify deployment on Windows.

14.3 Conclusion

The completed system demonstrates a modular approach to speaker identification. Four pretrained encoders are compared through independent downstream classifiers, while one desktop interface provides audio analysis, prediction, visualisation, settings and consolidated reporting. The current X-Vector configuration is the strongest measured baseline, and the architecture is ready for additional encoders and more rigorous evaluation.

15. References

Model descriptions are based on the official model cards and the project's configured pretrained repositories.

1. **SpeechBrain ECAPA-TDNN model card**
<https://huggingface.co/speechbrain/spkrec-ecapa-voxceleb>
2. **SpeechBrain X-Vector model card**
<https://huggingface.co/speechbrain/spkrec-xvect-voxceleb>
3. **Microsoft WavLM Base Plus SV model card**
<https://huggingface.co/microsoft/wavlm-base-plus-sv>
4. **Microsoft UniSpeech-SAT Base Plus SV model card**
<https://huggingface.co/microsoft/unispeech-sat-base-plus-sv>
5. **SpeechBrain project**
<https://speechbrain.github.io/>
6. **scikit-learn SVM documentation**
<https://scikit-learn.org/stable/modules/svm.html>
7. **CustomTkinter documentation**
<https://customtkinter.tomschimansky.com/>
8. **librosa documentation**
<https://librosa.org/doc/latest/>
9. **ReportLab documentation**
<https://docs.reportlab.com/>

Project-specific measurements in this document were taken from the local metrics, metadata, classifier artifacts and source code present on 24 July 2026.